

# Prompt: Make Kisan Setu Buyer & Lister Dashboards Fully Functional

I am working on **Kisan Setu**, an agricultural marketplace platform connecting farmers/listers directly with buyers.

I want you to make the **Buyer Dashboard** and **Farmer/Lister Dashboard** fully functional for a working demo. Do not redesign the entire website or unnecessarily change the existing UI. First inspect the existing codebase, routing, authentication, database/API structure, and current dashboard components, then implement the functionality within the existing design system.

## Demo Login Credentials

### Farmer / Lister

- Mobile: +919999999999
- Password: demo1234

### Buyer

- Mobile: +918888888888
- Password: demo1234

These demo accounts must work reliably for presentation/demo purposes.

---

## 1. Authentication

Implement working login functionality for both user types.

### Farmer/Lister Login

When the user logs in with:

+919999999999 / demo1234

authenticate them as a **Farmer/Lister** and redirect them to the Farmer/Lister Dashboard.

### Buyer Login

When the user logs in with:

+918888888888 / demo1234

authenticate them as a **Buyer** and redirect them to the Buyer Dashboard.

Requirements:

- Show proper validation for incorrect credentials.
  - Maintain login state after page refresh.
  - Provide logout functionality.
  - Prevent buyers from accessing farmer-only dashboard routes and vice versa.
  - If the project already has authentication, extend/fix it instead of creating a conflicting authentication system.
  - Do not expose passwords in the UI or source code unnecessarily outside the demo authentication mechanism.
- 

## 2. Farmer / Lister Dashboard

Make the farmer dashboard functional rather than just a static UI.

The farmer should be able to:

### Dashboard Overview

Display useful demo statistics such as:

- Total Listings
- Active Listings
- Products Sold
- Pending Orders
- Total Earnings
- Recent Orders
- Recent Listings

Use realistic demo data.

### Product Management

Create a working **Add Product/List Product** flow.

The farmer should be able to enter:

- Product name
- Category
- Description
- Quantity
- Unit
- Price
- Location
- Minimum order quantity
- Product image

After submitting:

- Product should appear in the farmer's listings.
- Product should be visible in the buyer marketplace.
- Show success feedback.
- Allow editing an existing listing.
- Allow deleting/deactivating a listing.
- Allow changing listing availability.

Example products can include:

- Rice
- Wheat
- Potatoes
- Tomatoes
- Mustard
- Onions
- Vegetables
- Fruits

## **Listings Page**

Create a functional listings table/grid containing:

- Product
- Category
- Quantity
- Price
- Status
- Date listed
- Actions

Actions:

- Edit
- Delete
- Activate/Deactivate
- View details

## **Orders**

The farmer should be able to see buyer orders.

Each order should contain:

- Order ID
- Buyer
- Product
- Quantity
- Total amount
- Order date
- Delivery location

- Order status

Implement order status updates such as:

Pending → Confirmed → Processing → Shipped → Delivered

Allow the farmer to update the status where appropriate.

## Earnings

Show:

- Total earnings
- Pending earnings
- Completed sales
- Recent transactions

Include a simple earnings overview/chart if the existing dashboard supports charts.

---

## 3. Buyer Dashboard

Make the buyer dashboard fully functional.

### Dashboard Overview

Show:

- Total Orders
- Active Orders
- Completed Orders
- Total Spending
- Favorite/Saved Products
- Recent Orders

Use realistic demo data.

### Marketplace

Create a functional marketplace where buyers can:

- Browse farmer products
- Search products
- Filter by category
- Filter by price
- Filter by location
- Sort products
- View product details

Each product should show:

- Product image
- Product name
- Farmer name
- Price
- Unit
- Available quantity
- Location
- Rating if available
- Add to Cart button

## **Product Details**

When a buyer opens a product:

- Show complete product information.
- Show farmer/lister information.
- Show available quantity.
- Allow quantity selection.
- Show calculated total price.
- Add to cart.
- Provide a clear checkout/buy option.

## **Cart**

Implement a working shopping cart.

Buyer should be able to:

- Add products.
- Change quantities.
- Remove products.
- See subtotal.
- See total.
- Proceed to checkout.

Cart state should persist during the session.

## **Checkout**

Create a functional demo checkout flow.

Collect/display:

- Buyer name
- Phone number
- Delivery address
- Product summary
- Quantity
- Total amount

For the demo, do **not** integrate a real payment gateway unless one already exists.

Instead, provide a **Demo Payment / Place Order** flow.

After placing an order:

- Generate an order ID.
  - Show order confirmation.
  - Add the order to the buyer's order history.
  - Make the order visible in the corresponding farmer's orders.
  - Clear the purchased items from the cart.
- 

## 4. Buyer Orders

Create a functional order history page.

Display:

- Order ID
- Product
- Farmer
- Quantity
- Amount
- Order date
- Delivery address
- Status

Allow the buyer to open an order and see its details.

Show an order progress indicator:

Order Placed → Confirmed → Processing → Shipped → Delivered

---

## 5. Buyer ↔ Farmer Data Connection

This is extremely important.

The dashboards should not behave like two completely separate static demos.

If a farmer creates a new product:

**Farmer Dashboard → Add Product → Marketplace → Buyer Dashboard**

The buyer should be able to see that product.

If a buyer purchases a farmer's product:

**Buyer Dashboard → Checkout → Order Created → Farmer Dashboard → Orders**

The farmer should be able to see the new order.

Use the project's existing backend/database/API if available.

If there is no backend/database currently implemented, create a clean demo data/state layer using the existing project architecture so that the interaction works consistently across the dashboards.

Do not hardcode separate contradictory data in each dashboard.

---

## 6. Notifications

Add basic notifications/activity updates.

Examples:

**Farmer receives:**

- "New order received"
- "Order #KS1024 has been placed"
- "Your product listing was published"

**Buyer receives:**

- "Order #KS1024 confirmed"
- "Your order has been shipped"
- "New products are available"

Use the existing notification system if present.

---

## 7. Demo Data

Populate the application with realistic demo data so that the dashboards look complete immediately after login.

Create:

- Several farmers
- Several products
- Several buyers/orders
- Order history
- Earnings/transactions
- Categories
- Product images/placeholders

The demo farmer account must have existing listings and orders.

The demo buyer account must have existing orders and saved/favorite products.

---

## 8. Navigation & Route Protection

Ensure the following works:

### Farmer

Login → Farmer Dashboard

Farmer navigation:

- Dashboard
- My Listings
- Add Product
- Orders
- Earnings
- Profile
- Logout

### Buyer

Login → Buyer Dashboard

Buyer navigation:

- Dashboard
- Marketplace
- Cart
- My Orders
- Favorites
- Profile
- Logout

Protect dashboard routes based on the authenticated user's role.

---

## 9. UI/UX Requirements

Keep the existing **Kisan Setu** visual identity.

Do not replace the current design unnecessarily.

Make sure:

- Buttons actually work.
- Forms actually submit.
- Tables update.



- Search/filter works.
- Modals work.
- Navigation works.
- Loading states are present where necessary.
- Empty states are handled.
- Success/error messages are displayed.
- Mobile responsiveness is preserved.
- No broken links or dead buttons remain.

Use the existing components, colors, typography, icons, and styling wherever possible.

---

## 10. Important Implementation Rules

Before making changes:

1. Inspect the entire existing project structure.
2. Identify the frontend framework.
3. Identify the backend/API/database.
4. Identify the existing authentication implementation.
5. Identify existing buyer/farmer dashboard components.
6. Reuse existing components and architecture wherever possible.
7. Do not unnecessarily rewrite working parts of the application.
8. Keep the code modular and maintainable.
9. Handle loading, error, and empty states.
10. Do not introduce unnecessary dependencies.

After implementation, test the complete flow.

### Required Demo Flow

Test this exact flow:

**Farmer** Login → Dashboard → Add Product → Product appears in Listings → Product appears in Marketplace

**Buyer** Login → Marketplace → Open Product → Add to Cart → Checkout → Place Demo Order

Then verify:

Buyer Order → Farmer Orders

Then test:

Farmer updates order status → Buyer sees updated order status

Also test:

- Logout
- Login again
- Refresh page while logged in
- Invalid login
- Protected routes
- Empty cart
- Product editing/deletion
- Search and filtering

Fix all errors encountered during testing.

## Final Requirement

The final result should feel like a **real working Kisan Setu agricultural marketplace**, not a collection of static dashboard screens.

The main objective is to make the **Buyer and Farmer/Listener dashboards completely functional for a live project demonstration**, while preserving the existing Kisan Setu UI and architecture.